Embedded Wallets for Shielded Bitcoin

Threshold Seed Custody, Proving Placement, and Side-Channel-Hardened PIN Authorization for Replay-Based Shielded Transfers

Roi Mateo
roimateo.nsofts@gmail.com

Kiley Kunze Zhai
zhai16263@gmail.com

Suzuki Motoyasu
suzukimotoyasu9429@outlook.com

November 2026

Abstract

Shielded Bitcoin is a recently proposed metaprotocol for private Bitcoin-denominated transfers on the Bitcoin L1: value is held in encrypted notes, spends publish nullifiers, validity is established by a zero-knowledge proof, and the shielded state is reconstructed by deterministic replay of Bitcoin-published envelopes. Its specification assumes a self-contained wallet, whereas mainstream adoption increasingly runs through embedded wallets, whose keys live inside an app, are unlocked with a PIN or biometric, and are backed up by a provider. We ask how to build such a wallet without turning the provider into a custodian or a surveillance point. Two obstacles arise: spend authority is a scalar consumed as a proof witness rather than a signing key, so threshold-signature designs do not apply; and the PIN that gates key release leaks timing information to any Android app through the battery fuel gauge. We present SBEW, which (i) splits the wallet seed 2-of-3 across a hardware-bound device share, a provider share released by an attempt-limited oblivious-PRF PIN check, and a recovery share; (ii) defines on-device, enclave, and collaborative proving with their trust and leakage; (iii) replaces single-indexer trust with k-of-n root agreement and delta-download synchronization; (iv) hardens PIN entry; and (v) wraps the device share on Cortex-M chips with the tweakable block cipher Rotom. We prove non-custody, PIN-leak tolerance, theft resistance, sync integrity, recovery, and the security of the Rotom wrap by reduction to the Shielded Bitcoin security theorem and standard assumptions. We also prove that randomized keypads leak at most a PIN's digit-repetition pattern, capping ten-guess success on 4-digit PINs at 1.5%, and at the blind rate for distinct digits; in simulation, fixed keypads sampled at 10 Hz raise it from 0.1% to 1.2–6.7%. Finally, a provider that both scans incoming notes and relays envelopes can reconstruct its users' payment graph. SBEW requires no change to the Shielded Bitcoin protocol.

Keywords: Shielded Bitcoin, embedded wallets, threshold secret sharing, oblivious PRF, zero-knowledge proofs, side channels, fuel gauge, tweakable block ciphers, Rotom

1 Introduction

Bitcoin's transaction graph is public. Shielded Bitcoin [1] proposes shielded transfers that are published on Bitcoin L1 without consensus changes: a sender publishes a transfer *envelope* carrying nullifiers, recipient ciphertexts, a sender-recovery ciphertext, and a succinct proof; indexers replay accepted envelopes in Bitcoin order to maintain a note tree, a nullifier set, and block-level roots. Its key hierarchy derives spend, nullifier, incoming-viewing, and outgoing-viewing capabilities from one seed, and its disclosure model guarantees that no read capability confers spend authority [1, §6–7].

The Shielded Bitcoin specification is deliberately scoped to the transfer layer. It notes that a prover separated from the wallet "is inside the sender's trusted computing base" [1, §10], that a wallet which does not replay treats its indexer as a trusted component (assumption A11) [1,

§17], and it leaves wallet policy, fee publication, and network observation as future work [1, §22]. These are precisely the components that an *embedded wallet*—a wallet whose keys are provisioned inside a consumer app and unlocked with a short secret—must decide.

Embedded wallets are attractive because they remove seed-phrase handling from the user, but their security rests on three things that interact badly with a shielded protocol:

1. **Key custody without a signature.** Current embedded wallets typically split a signing key with threshold ECDSA or Schnorr [3, 4] or keep it in a provider enclave. In Shielded Bitcoin, however, spending is authorized by proving knowledge of $\mathsf{sk}_{\mathsf{spend}}$ inside the transfer circuit, from which the circuit itself derives $\mathsf{sk}_{\mathsf{nf}}$ and the diversified transmission key [1, §9, §14]. There is no signature to thresholdize.
2. **Short-secret authorization on a leaky device.** The PIN that releases key material is entered on a phone. Giraud and Naccache [2] showed that on Android 9–12 any application can sample the battery fuel gauge's instantaneous current through `BatteryManager` without user-visible permission, detect individual key presses, and use inter-press timing to narrow the PIN candidates entered in another application.
3. **A provider who sees a lot.** The provider authenticates the user, may scan for incoming notes, and may relay envelopes. Each role is benign alone; combined, they can undo the transfer-layer privacy that Shielded Bitcoin provides.

Contributions.

- A system and threat model for embedded Shielded Bitcoin wallets with five adversary classes, including a co-resident malicious app exploiting the fuel-gauge channel (§3).
- SBEW, a method that maps every Shielded Bitcoin capability to a custody location, splits $\mathsf{seed}_{\mathsf{wallet}}$ 2-of-3 with an attempt-limited OPRF PIN gate, and defines three proving placements with explicit trust and leakage (§4).
- A synchronization procedure that replaces single-indexer trust (A11) with k-of-n root agreement and privacy-preserving delta download (§5).
- Game-based definitions and full proofs for non-custody, PIN-leak tolerance, device theft, sync integrity, and recovery; an exact, tight guessing bound for randomized keypads (Theorem 4); and a proof that a provider using V1 with authenticated relaying reconstructs its users' payment graph (§6).
- A hardware-wallet variant that wraps the device share with Rotom-256 [10] under structured tweaks, with a proof of privacy and integrity (Theorem 7), a bound on the chosen-input power traces an attacker can collect (Proposition 2), and cycle costs derived from the Rotom implementation figures (§4.7, §6.8).
- A quantitative model of fuel-gauge PIN leakage and of the effect of keypad randomization (§7).

SBEW changes nothing in the Shielded Bitcoin transfer relation, envelope, or replay rules; it is a wallet-layer and deployment-profile construction. A protocol-level alternative that would enable threshold signing is discussed separately (§8).

2 Background

2.1 Shielded Bitcoin wallet capabilities

Table 1 summarizes the Shielded Bitcoin key hierarchy [1, Table 1]. All capabilities are derived from a uniformly random 32-byte $\mathsf{seed}_{\mathsf{wallet}}$ with domain-separated HKDF-SHA256 labels; the specification states that low-entropy passwords are not suitable seeds [1, App. A.2]. A payment address is (d, pk_d) with $\mathsf{pk}_d = [\mathsf{sk}_{\mathsf{view}}]G_d$ and $\mathsf{sk}_{\mathsf{view}}$ derived from $\mathsf{vk}_{\mathsf{in}}$.

A spend proves, for each input, Merkle membership under a replayed anchor root $R[h_{\mathsf{anchor}}]$, correct nullifier derivation $\mathsf{nf} = H_{\mathsf{nf}}(\mathsf{sk}_{\mathsf{nf}}, \rho, \mathsf{pos})$, and the ownership rule that re-derives pk_d from

Object	Derivation [1]	Capability
$\mathsf{sk}_{\mathsf{master}}$	$\mathrm{HKDF}_{\mathsf{master}}(\mathsf{seed}_{\mathsf{wallet}})$	root secret
$\mathsf{sk}_{\mathsf{spend}}$	$\mathrm{ScalarFromBytes}(\mathrm{HKDF}_{\mathsf{spend}}(\mathsf{sk}_{\mathsf{master}}))$	spend authority (proof witness)
$\mathsf{sk}_{\mathsf{nf}}$	$\mathrm{HKDF}_{\mathsf{nf}}(\mathsf{sk}^{\mathsf{ser}}_{\mathsf{spend}})$, re-derived in circuit	nullifier derivation
$\mathsf{vk}_{\mathsf{in}}$	$\mathrm{HKDF}_{\mathsf{in}}(\mathsf{sk}^{\mathsf{ser}}_{\mathsf{spend}})$	incoming detection, read-only
$\mathsf{vk}_{\mathsf{out}}$	$\mathrm{HKDF}_{\mathsf{out}}(\mathsf{sk}_{\mathsf{master}})$	sent-note recovery, read-only

Table 1: Shielded Bitcoin wallet capabilities. Only $\mathsf{sk}_{\mathsf{spend}}$ (and its ancestors) authorizes spends.

$\mathsf{sk}_{\mathsf{spend}}$; it also proves ciphertext consistency and value conservation, and binds the whole body through h_{body} [1, §13–14]. The witness contains note plaintexts, Merkle paths, and a single $\mathsf{sk}_{\mathsf{spend}}$ for all inputs [1, §14.3]. The reference profile uses Groth16, a 2-input/2-output envelope of 610 bytes in one `OP_RETURN` output, and an anchor window of W=100, $K_{\min}$=1 blocks [1, App. A].

Two specification statements constrain any wallet architecture. A delegated prover is in the sender's TCB because it sees the witness [1, §10]. A wallet that does not verify replay must trust its indexer for the claimed prefix (A11) [1, §17]. The privacy theorem covers the accepted-transfer transcript; fee linkage, network observation, and wallet policy are outside it and are characterized as additional leakage [1, §20].

2.2 Embedded wallets

We use *embedded wallet* for any wallet whose key material is created and used inside a host application, unlocked with a short user secret (PIN, passkey, biometric), and recoverable through a provider-assisted flow without a user-managed seed phrase. Deployed designs combine three ingredients: hardware-bound device keys (Android Keystore/StrongBox, Apple Secure Enclave); provider-held key shares, either for threshold signing [3, 4] or released after short-secret authentication; and attempt-limited PIN verification in an HSM or enclave, typically built on an oblivious PRF so that the server never sees the PIN [5]. Attempt limiting is what makes a 4- or 6-digit PIN meaningful: an attacker without the provider's cooperation gets at most L online guesses.

2.3 The fuel-gauge side channel

Giraud and Naccache [2] observe that high-end Android devices delegate battery accounting to a dedicated fuel-gauge IC whose `CURRENT_NOW` reading is exposed to every application via `BatteryManager`, with no SELinux, framework, or runtime restriction on Android 12 and earlier. The register refreshes at roughly 4–10 Hz, too slow for cryptographic side channels but fast enough for human input. A foreground service (permission granted without a prompt) can sample in the background. Touches are visible as current spikes, sharply so when haptic feedback is enabled. Using the time between presses and the known position of the confirmation key, the authors enumerate PINs consistent with the observed delays by walking backwards from the validation key. They do not quantify success rates. For an embedded wallet, this channel attacks exactly the secret that gates key release.

3 System and Threat Model

Parties. The user's device $\mathcal{D}$ runs the host app and has a hardware keystore offering non-exportable, app-scoped keys with optional user-presence (biometric) gating. In the hardware variant (§4.7), the device share is instead held by a microcontroller with a chip-unique key. The provider $\mathcal{P}$ authenticates the user, runs an HSM or attested enclave that stores one seed

share and enforces a PIN attempt counter, and may optionally offer note scanning, proving, and relaying. Indexers $\mathcal{I}_1, \ldots, \mathcal{I}_n$ replay Shielded Bitcoin and serve roots, leaves, and nullifiers; they are operated by parties independent of $\mathcal{P}$ and of each other. A recovery channel $\mathcal{R}$ (a passkey-protected backup or offline card) holds the third share.

Adversaries.

$\mathsf{A}_{\mathsf{app}}$ — **co-resident malicious app.**
Runs on $\mathcal{D}$ under a different UID, can sample the fuel gauge and other zero-permission sensors, cannot read other apps' memory or use their keystore keys.

$\mathsf{A}_{\mathsf{thief}}$ — **device thief.**
Physically holds an unlocked or locked $\mathcal{D}$; may also hold a fuel-gauge trace collected earlier by $\mathsf{A}_{\mathsf{app}}$.

$\mathsf{A}_{\mathsf{prov}}$ — **malicious or breached provider.**
Controls $\mathcal{P}$'s storage and logic outside the HSM. It does not collude with the device or with the holder of the recovery share: any two shares reconstruct the seed, so no 2-of-3 scheme can tolerate such collusion.

$\mathsf{A}_{\mathsf{idx}}$ — **dishonest indexers.**
Up to t of the n indexers return arbitrary roots, omit data, or log queries.

$\mathsf{A}_{\mathsf{net}}$ — **network observer.**
Observes Bitcoin, the mempool, and traffic metadata between $\mathcal{D}$ and other parties.

Goals.

E1 Non-custody.
$\mathcal{P}$ alone, or $\mathcal{P}$ with any read capability, cannot spend.

E2 PIN-leak tolerance.
Learning the PIN exactly (e.g. via $\mathsf{A}_{\mathsf{app}}$) does not by itself allow spending.

E3 Recovery.
Loss of any one of $\{\mathcal{D}, \mathcal{P}, \mathcal{R}\}$ does not lose funds; the wallet state is rebuilt by Shielded Bitcoin seed recovery [1, §16.4].

E4 Bounded leakage.
Each deployment mode's leakage is $\mathsf{Leak}_{\mathsf{SB}} \cup \mathsf{Leak}_{\mathsf{mode}}$ with $\mathsf{Leak}_{\mathsf{mode}}$ stated explicitly.

E5 Sync integrity.
A wallet that cannot replay still spends only against $R[h] = \mathrm{Replay}(\mathrm{Env}(C))[h]$ unless more than t indexers collude.

4 The SBEW Method

4.1 Capability placement

Table 2 fixes where each Shielded Bitcoin object resides. The organizing rule is that *spend-capable material exists in reconstructed form only on the device, only for the duration of one proof*, while read capabilities are placed according to the chosen privacy mode.

$\mathsf{vk}_{\mathsf{in}}$ and $\mathsf{vk}_{\mathsf{out}}$ are derived once at setup and cached under a keystore key so that scanning and history display do not require seed reconstruction. This is safe for E1 because, by Shielded Bitcoin assumption A5, neither viewing key yields spend authority [1, §17].

4.2 Seed sharing and PIN-gated release

Let $s = \mathsf{seed}_{\mathsf{wallet}} \in \{0,1\}^{256}$. At setup the device samples s, computes Shamir shares $(\sigma_{\mathcal{D}}, \sigma_{\mathcal{P}}, \sigma_{\mathcal{R}}) \leftarrow \mathsf{Share}_{2,3}(s)$ over $\mathrm{GF}(2^8)$ bytewise [6], and erases s.

Object	At rest	In use
$\mathsf{seed}_{\mathsf{wallet}}$	never stored whole; 2-of-3 shares $\sigma_{\mathcal{D}}, \sigma_{\mathcal{P}}, \sigma_{\mathcal{R}}$	reconstructed in app memory for one spend, then zeroized
$\mathsf{sk}_{\mathsf{spend}}, \mathsf{sk}_{\mathsf{nf}}$	not stored	derived from $\mathsf{seed}_{\mathsf{wallet}}$; passed to prover (M1) or secret-shared (M3)
$\mathsf{vk}_{\mathsf{in}}$	device keystore-wrapped; optionally escrowed to scanner (mode V1)	note detection
$\mathsf{vk}_{\mathsf{out}}$	device keystore-wrapped; never delegated	sent-note recovery
note plaintexts, positions, paths	device cache (non-critical [1, §16])	witness assembly

Table 2: SBEW placement of Shielded Bitcoin capabilities.

Device share. $\sigma_{\mathcal{D}}$ is stored as $\mathsf{AEAD}_{K_{\mathcal{D}}}(\sigma_{\mathcal{D}})$, where $K_{\mathcal{D}}$ is a non-exportable keystore key bound to the app's signing identity and, in the recommended profile, to user presence (biometric or device credential).

Provider share and PIN gate. The user chooses a PIN π. The device and the provider HSM run an OPRF [5]: the device learns $y = F_{k_u}(\pi)$ without revealing π; the HSM holds k_u and a counter $c_u \le L$. The device derives $\tau = \mathrm{HKDF}_{\mathsf{auth}}(y)$ and registers τ with the HSM, which stores $(k_u, \tau, \sigma_{\mathcal{P}}, c_u{=}0)$. To unlock:

1. $\mathcal{D}$ blinds π; the HSM evaluates under k_u only if $c_u < L$ and increments c_u.
2. $\mathcal{D}$ unblinds to y', sends $\tau' = \mathrm{HKDF}_{\mathsf{auth}}(y')$ with a fresh device-attested ephemeral key epk.
3. If $\tau' = \tau$ the HSM resets c_u and returns $\mathsf{Enc}_{\mathsf{epk}}(\sigma_{\mathcal{P}})$; otherwise it returns failure. After L consecutive failures the share is locked and only the $\{\sigma_{\mathcal{D}}, \sigma_{\mathcal{R}}\}$ path remains.

The PIN therefore never protects s by itself—consistent with Shielded Bitcoin's requirement that seeds are full-entropy—and it is never the only factor: release of $\sigma_{\mathcal{P}}$ is useless without $\sigma_{\mathcal{D}}$, which requires the device and its keystore. We do not rely on the PIN for confidentiality against $\mathcal{P}$ itself (the HSM holds k_u and could test PINs); non-custody against $\mathcal{P}$ follows from the threshold, not from the PIN.

Recovery share. $\sigma_{\mathcal{R}}$ is encrypted under a passkey-derived key and stored in the user's cloud backup, or printed. Device loss: $\{\sigma_{\mathcal{P}}, \sigma_{\mathcal{R}}\}$; provider loss: $\{\sigma_{\mathcal{D}}, \sigma_{\mathcal{R}}\}$. After recovery the device re-shares a *fresh* split of the same s (the seed, and hence every address and note, is unchanged). A fingerprint φ of the first payment address is kept with $\sigma_{\mathcal{D}}$ and $\sigma_{\mathcal{R}}$ (not at $\mathcal{P}$) so that a corrupted share is detected at reconstruction (Theorem 6).

4.3 Proving placement

Because the witness contains $\mathsf{sk}_{\mathsf{spend}}$, all note plaintexts, and Merkle paths [1, §14.3], the choice of prover determines both custody and privacy. SBEW defines three placements.

M1 — on-device proving (default). $\mathcal{D}$ reconstructs s, derives $\mathsf{sk}_{\mathsf{spend}}$, assembles the witness, runs the prover, and zeroizes. No other party learns any witness value. The cost is mobile proving time and memory for the deployed circuit; we do not report figures because the Shielded Bitcoin circuit parameters are not yet fixed [1, §22.1]. The residual risk is a compromise of $\mathcal{D}$ *during* a spend, which is inherent to any hot wallet.

M2 — attested-enclave proving. $\mathcal{D}$ sends the full witness to a provider enclave whose measurement it verifies by remote attestation. This matches the specification's "external proving service" [1, App. A.1] and places the enclave in the TCB [1, §10]. If enclave isolation fails,

the provider learns $\mathsf{sk}_{\mathsf{spend}}$ (violating E1) and the complete history of the spent notes. M2 is therefore a performance fallback with an explicit hardware trust assumption, not a non-custodial mode.

M3 — collaborative proving. $\mathcal{D}$ and $\mathcal{P}$ run a two-party collaborative zk-SNARK [7] in which $\mathsf{sk}_{\mathsf{spend}}$ and the rest of the witness are additively secret-shared and neither party sees them in the clear; the output is an ordinary proof that indexers verify unchanged. In SBEW the device derives $\mathsf{sk}_{\mathsf{spend}}$ locally and shares it for the session; a stronger variant derives the shares of $\mathsf{sk}_{\mathsf{spend}}$ directly from $\sigma_{\mathcal{D}}, \sigma_{\mathcal{P}}$ inside the MPC so that s is never reconstructed, at the price of evaluating the HKDF chain of Table 1 in MPC. M3 preserves E1 even against a breached enclave, but adds interaction and communication; its cost for the Shielded Bitcoin circuit is an open measurement.

4.4 Scanning modes

Detection uses $\mathsf{vk}_{\mathsf{in}}$ to trial-decrypt every output [1, §16.1].

V0 — local scan.
$\mathcal{D}$ downloads each block's output fields $(h_{\mathsf{body}}, j, \mathsf{pk}_{\mathsf{eph}}, \mathsf{ct}^{\mathsf{note}})$ and trial-decrypts. No party learns which outputs are the user's.

V1 — delegated scan.
$\mathsf{vk}_{\mathsf{in}}$ is shared with a scanning service, which notifies $\mathcal{D}$. This is exactly the Shielded Bitcoin IncomingView disclosure: a broad read capability that the audit scope does not cryptographically restrict [1, §7]. The scanner learns every incoming note's value, diversifier, and timing for the wallet.

Shielded Bitcoin has no detection-only key weaker than $\mathsf{vk}_{\mathsf{in}}$. A fuzzy-message-detection tag [8] placed through the reserved h_{aux} slot [1, §22.5] could give a scanner a tunable false-positive rate instead of exact knowledge; we leave this extension open because it changes leaf derivation for participating notes.

4.5 Publication and fees

Shielded Bitcoin carrier transactions must be funded on L1, and funding them from the user's transparent wallet links envelopes at the Bitcoin layer [1, §20.5]. An embedded provider is a natural sponsor and relayer: $\mathcal{D}$ hands $\mathcal{P}$ the finished envelope, and $\mathcal{P}$ wraps it in a carrier funded from a pooled wallet. The provider learns nothing that is not on chain about the envelope itself, but it learns the mapping from the *authenticated account* to the envelope and its timing. Section 6.9 shows why this matters. SBEW therefore recommends submitting envelopes through an anonymous channel (e.g. Tor) to a relayer that does not authenticate the user, with the provider paying fees through unlinkable prepaid tokens, or the PIPE-based fee vault anticipated by Shielded Bitcoin [1, §20.5] once specified.

4.6 PIN-entry hardening

The method fixes four UI rules derived from the fuel-gauge channel and evaluated in §7: (R1) re-randomize the digit layout and the position of the confirmation key on every entry; (R2) disable haptic feedback on the PIN pad, since vibration makes presses trivially visible [2]; (R3) require 6 digits, or reject PINs with repeated digits, because randomization does not hide repetitions; (R4) never use the PIN as the sole factor (it releases $\sigma_{\mathcal{P}}$ only).

4.7 Hardware-wallet variant: wrapping the device share with Rotom

Phone keystores are not available on every platform, and some users want the device share on a dedicated chip rather than on a general-purpose phone. In the hardware variant, $\sigma_{\mathcal{D}}$ is

held by a 32-bit microcontroller in a hardware wallet or companion device, typically an ARM Cortex-M3/M4 class part. The chip stores the share in external or general flash, wrapped under a chip-unique 256-bit key K_{HW} that never leaves the chip (OTP, protected flash, or PUF-derived). We instantiate the wrap with Rotom [10], a 128-bit tweakable block cipher (TBC) with a 128-bit tweak designed for exactly this class of microcontroller. We use the full variant Rotom-256, which claims security up to time 2^{240} for data up to 2^{128} blocks in the single- and related-tweak settings [10, §2.4].

Why a TBC, and why this construction. The payload is short and fixed: one 32-byte share, which is two 128-bit blocks. The chip also already keeps a monotone epoch counter for re-sharing. A tweakable block cipher lets every block of every wrap use its own tweak, so that each call behaves as an independent random permutation [11]. This gives a three-call wrap whose security we prove directly from the TBC assumption (Theorem 7), with no nonce generation on the chip and no reliance on mode-level bounds. Rotom's authors also instantiate a ΘCB3-style AEAD [12] on the same cipher [10, App. D]. That AEAD is a reasonable alternative, but it needs a unique nonce per wrap, and its security argument is that of the OCB/ΘCB3 hybrid the authors describe.

Tweak encoding. Every Rotom call uses a 128-bit tweak with an injective, fixed-width layout:

$$T(\mathsf{dom}, \mathsf{idx}) = \underbrace{\mathsf{dom}}_{8} \parallel \underbrace{\mathsf{idx}}_{8} \parallel \underbrace{\mathsf{ver}}_{16} \parallel \underbrace{\mathsf{devid}}_{64} \parallel \underbrace{\mathsf{epoch}}_{32},$$

with $\mathsf{dom} \in \{\mathtt{enc}, \mathtt{tag}\}$ and $\mathsf{idx} \in \{1, 2\}$. The context $\mathsf{ctx} = (\mathsf{ver}, \mathsf{devid}, \mathsf{epoch})$ is chip-internal state: an attacker can submit blobs but cannot choose the context. Because the layout is a plain concatenation of fixed-width fields, distinct $(\mathsf{dom}, \mathsf{idx}, \mathsf{ctx})$ give distinct tweaks; no hash, and therefore no tweak collision, is involved.

The wrap RW. Write $\widetilde{E}_K^T$ for Rotom-256 under key K and tweak T. Let $\sigma_{\mathcal{D}} = \sigma_1 \| \sigma_2$ with $\sigma_1, \sigma_2 \in \{0,1\}^{128}$.

- $\mathsf{RW.Wrap}_K(\mathsf{ctx}, \sigma_1 \| \sigma_2)$ outputs (C_1, C_2, Γ), where

$$C_1 = \widetilde{E}_K^{T(\mathtt{enc},1)}(\sigma_1), \qquad C_2 = \widetilde{E}_K^{T(\mathtt{enc},2)}(\sigma_2), \qquad \Gamma = \widetilde{E}_K^{T(\mathtt{tag},1)}(\sigma_1 \oplus \sigma_2).$$

- $\mathsf{RW.Unwrap}_K(\mathsf{ctx}, C_1, C_2, \Gamma)$ computes $\sigma_i' = \widetilde{D}_K^{T(\mathtt{enc},i)}(C_i)$ for $i = 1, 2$, and returns $\sigma_1' \| \sigma_2'$ if

$$\widetilde{D}_K^{T(\mathtt{tag},1)}(\Gamma) = \sigma_1' \oplus \sigma_2',$$

  and $\perp$ otherwise.

The blob is 48 bytes. The chip enforces three rules: (W1) each context is wrapped at most once, which the monotone epoch guarantees; (W2) Unwrap runs only after on-chip user confirmation; and (W3) the chip erases K_{HW} after F cumulative $\perp$ results, with $F = 10$ by default. After a successful unwrap the chip releases $\sigma_{\mathcal{D}}$ to the wallet over an authenticated channel, or keeps it on the chip if the chip also performs seed reconstruction.

Cost. Wrapping uses three Rotom encryptions and unwrapping three decryptions, each with its own tweakey schedule because the tweak changes. From the per-block figures in [10, Table 11] (tweakey schedule plus decryption), Unwrap costs $3 \times (780 + 852) = 4{,}896$ cycles on Cortex-M4 and $3 \times (836 + 908) = 5{,}232$ cycles on Cortex-M3 for the compact non-unrolled implementation, or 4,527 and 4,863 cycles fully unrolled. That is under 0.1 ms at 64 MHz. The non-unrolled tweakey schedule and decryption code reported there total about 1.15 KB of ROM on Cortex-M4, with about 40 bytes of stack. These figures are summed from separately reported components, not measured for RW as a whole.

Implementation side channels. Rotom admits constant-time implementations on Cortex-M3/M4 [10, §5.4.1]. An unmasked implementation, however, is vulnerable to correlation power analysis: in the designers' simulation, the whitening key and first-round key were recovered within about 500 traces with varying known inputs [10, §5.4.2]. Rule W3 limits the number of such traces an attacker can obtain (Proposition 2). Masking with linear gadgets and A2B/B2A conversion, which Rotom supports without masked field multipliers [10, §5.4.2], should still be used where physical attacks are in scope.

5 Synchronization without Single-Indexer Trust

An embedded wallet usually cannot replay the Bitcoin chain. Shielded Bitcoin's fallback is A11: trust the selected indexer [1, §17]. A dishonest indexer cannot make a false root consistent with replay, but it can serve one to a wallet that does not check, causing spends that replay later rejects or, worse, a balance view that hides received notes. SBEW replaces A11 with agreement.

Root agreement. For the anchor height h the wallet intends to use, it queries n independent indexers for $R[h]$ and the block hash at h, and accepts r only if at least k responses agree on both and the block hash matches the wallet's own header chain (obtained by SPV). If no value reaches k, the wallet waits or falls back to replay of the recent window. It then checks every Merkle path for its inputs against r locally.

Theorem 5 in §6 proves that a false root is accepted only if at least k indexers collude; with independent operators this is a binomial tail (e.g. 8.6×10^{-3} for n=5, k=3, p=0.1, against $p = 0.1$ for one trusted indexer). Independence of operators is an assumption that should be justified by deployment, not presumed.

Private retrieval. Requesting Merkle paths for specific positions, or asking whether specific nullifiers are spent, tells the indexer which notes the wallet cares about—linking them to the wallet's network identity. SBEW retrieves *deltas* instead: for each new block the wallet downloads all appended leaves and all newly accepted nullifiers, maintains its own frontier of the append-only tree, and computes its own paths and spent status. Leaves and nullifiers are 32 bytes each [1, App. A.4], so a block with m accepted 2-to-2 transfers costs $128m$ bytes of state delta plus output ciphertexts for V0 scanning. Delta download makes every wallet's request identical for a given block, so indexers learn nothing beyond the wallet's online times.

6 Formal Security Analysis

This section states each goal of §3 as a security experiment and proves it by reduction to the assumptions of Shielded Bitcoin [1, §17–19] and to standard properties of the added primitives. We follow the conventions of [1, §19]: λ is the security parameter, adversaries are PPT, $q = q(\lambda)$ bounds the accepted envelopes in the output prefix, and negl denotes a negligible function. Proofs are given in full; where a step is an existing Shielded Bitcoin result we cite it rather than re-prove it.

6.1 Primitives, assumptions, and notation

Shamir sharing. Fix distinct non-zero evaluation points $x_{\mathcal{D}}, x_{\mathcal{P}}, x_{\mathcal{R}} \in \mathrm{GF}(2^8)$. For $s = (s_1, \ldots, s_{32}) \in \mathrm{GF}(2^8)^{32}$, $\mathsf{Share}_{2,3}(s)$ samples $a = (a_1, \ldots, a_{32})$ uniformly and outputs $\sigma_j = (s_i + a_i x_j)_{i=1}^{32}$ for $j \in \{\mathcal{D}, \mathcal{P}, \mathcal{R}\}$. Any two shares $\sigma_j, \sigma_{j'}$ reconstruct $s_i = \lambda_j \sigma_{j,i} + \lambda_{j'} \sigma_{j',i}$ with Lagrange coefficients $\lambda_j = x_{j'}/(x_j + x_{j'})$, $\lambda_{j'} = x_j/(x_j + x_{j'})$ (characteristic 2, so subtraction is addition). Each re-sharing (§4) samples a fresh a.

Fingerprint. At setup the device computes $\varphi = H_{\mathsf{fp}}(\mathsf{addr}_0)$, where addr_0 is the wallet's first diversified payment address, and stores φ next to $\sigma_{\mathcal{D}}$ and inside the encrypted $\sigma_{\mathcal{R}}$ backup, but *not* at $\mathcal{P}$. After any reconstruction the wallet re-derives addr_0 and accepts only if the fingerprint matches.

Keystore (KS). $K_{\mathcal{D}}$ is reachable only through an oracle $\mathcal{O}_{\mathsf{KS}}$ that answers requests from the wallet application's identity, and, when presence-bound, only after a user-presence event. No other party learns $K_{\mathcal{D}}$ or obtains $\mathcal{O}_{\mathsf{KS}}$ answers. The AEAD used for wrapping is IND-CPA secure with advantage $\mathrm{Adv}^{\mathsf{ind}}_{\mathsf{AEAD}}$.

PIN gate (GATE). We model the OPRF-based unlock of §4 by an ideal functionality $\mathcal{F}_{\mathsf{gate}}$ holding $(\pi, \sigma_{\mathcal{P}}, c, L)$. On $\mathsf{Test}(x)$ from a client: if $c \geq L$ return $\perp$; if $x = \pi$ set $c \leftarrow 0$ and return $\sigma_{\mathcal{P}}$; otherwise set $c \leftarrow c + 1$ and return fail. The real protocol realizes $\mathcal{F}_{\mathsf{gate}}$ against clients with error $\mathrm{Adv}^{\mathsf{oprf}}$, by the UC security of OPRF-based password protocols [5]. It is *oblivious*: the HSM's view of an honest unlock can be simulated without π up to $\mathrm{Adv}^{\mathsf{obl}}$.

Side channel (SC). Fuel-gauge and sensor observations are a randomized function of the PIN, the keypad layout, the user's motor behaviour, and time only; they are independent of s given these. This matches [2], who note that the 4–10 Hz refresh rate rules out attacks on cryptographic computation.

Collaborative proving (CO). The two-party prover Π_{co} of mode M3 has a simulator $\mathcal{S}_{\mathsf{co}}$ that, given the corrupted provider's input and the output proof, produces a view indistinguishable from the real one with advantage $\mathrm{Adv}^{\mathsf{co}}$ [7].

Shielded Bitcoin terms. $\mathrm{Adv}^{\mathsf{ks}}_{R_{\mathsf{tr}}}$ is the knowledge-soundness advantage (A3) and $\mathrm{Exp}^{\mathsf{bal}}$, $\mathrm{Adv}^{\mathsf{auth}}$ are the ledger-balance experiment and theft term of [1, Def. 5, Cor. 2]. We write $\mathrm{Exp}^{\mathsf{bal}+}$ and $\mathrm{Adv}^{\mathsf{auth}+}$ for the same experiment in which the adversary additionally receives $\mathsf{vk}_{\mathsf{in}}$ of the honest seed; assumption A5 states that spending authority cannot be recovered from disclosed $\mathsf{vk}_{\mathsf{in}}$, so $\mathrm{Adv}^{\mathsf{auth}+}$ is negligible under the same assumptions as $\mathrm{Adv}^{\mathsf{auth}}$.

Lemma 1 (Multi-epoch single-share privacy)**.** *Let s be arbitrary and let $\sigma^{(1)}_{\mathcal{P}}, \ldots, \sigma^{(r)}_{\mathcal{P}}$ be $\mathcal{P}$'s shares from r independent executions of $\mathsf{Share}_{2,3}(s)$. Then $(\sigma^{(1)}_{\mathcal{P}}, \ldots, \sigma^{(r)}_{\mathcal{P}})$ is uniform on $(\mathrm{GF}(2^8)^{32})^r$ and independent of s and of every other random variable not derived from the sharing coins.*

Proof. Fix an epoch e and byte i. The map $a_i^{(e)} \mapsto s_i + a_i^{(e)} x_{\mathcal{P}}$ is a bijection of $\mathrm{GF}(2^8)$ because $x_{\mathcal{P}} \neq 0$, so $\sigma^{(e)}_{\mathcal{P},i}$ is uniform whatever the value of s_i. The coins $a_i^{(e)}$ are independent across i and e and independent of everything else, so the $32r$ bytes are independent and uniform, and their joint distribution does not depend on s. □

6.2 Non-custody (E1)

Definition 1 (Non-custody experiment)**.** $\mathrm{Exp}^{\mathsf{nc}}_{\mathcal{A},\mathsf{M}}(\lambda)$ *for mode $\mathsf{M} \in \{\mathrm{M1}, \mathrm{M3}\}$ and scanning mode V: the challenger runs Shielded Bitcoin setup, samples s and a PIN π, and runs SBEW setup. $\mathcal{A}$ receives* pp*, all provider shares $\sigma^{(e)}_{\mathcal{P}}$, the HSM state (including the OPRF key and τ), and, if $\mathsf{V} = \mathrm{V1}$, $\mathsf{vk}_{\mathsf{in}}$. $\mathcal{A}$ may query an honest-user oracle $\mathcal{O}_U$ that performs a complete spend of $\mathcal{A}$'s choice: $\mathcal{A}$ plays the provider in the unlock, receives the finished envelope for relaying, and in M3 plays the provider in Π_{co}. $\mathcal{A}$ controls all other envelopes and outputs a prefix C. The experiment returns 1 iff replay is defined on* $\mathrm{Env}(C)$ *and some accepted envelope spends a note owned by s that $\mathcal{O}_U$ did not produce.*

Theorem 1 (Non-custody)**.** *For every PPT $\mathcal{A}$ making q_s oracle spends there is a PPT $\mathcal{B}$ with*

$$\Pr[\mathrm{Exp}^{\mathsf{nc}}_{\mathcal{A},\mathrm{M1}} = 1] \le q{\cdot}\mathrm{Adv}^{\mathsf{ks}}_{R_{\mathsf{tr}}} + \mathrm{Adv}^{\mathsf{auth}+}_{\mathcal{B}}, \qquad \Pr[\mathrm{Exp}^{\mathsf{nc}}_{\mathcal{A},\mathrm{M3}} = 1] \le q_s{\cdot}\mathrm{Adv}^{\mathsf{co}} + q{\cdot}\mathrm{Adv}^{\mathsf{ks}}_{R_{\mathsf{tr}}} + \mathrm{Adv}^{\mathsf{auth}+}_{\mathcal{B}}.$$

Proof. We use a sequence of games. Let W_i be the event that the experiment returns 1 in Game i.

Game 0 is $\mathrm{Exp}^{\mathsf{nc}}_{\mathcal{A},\mathsf{M}}$.

Game 1 replaces every $\sigma^{(e)}_{\mathcal{P}}$ given to $\mathcal{A}$ by an independent uniform string. The sharing coins are used nowhere except to compute the shares, and $\mathcal{A}$ sees only $\mathcal{P}$'s shares, so by Lemma 1 $\mathcal{A}$'s view has the same distribution in both games: $\Pr[W_1] = \Pr[W_0]$.

Game 2 (M3 only) replaces, one spend at a time, $\mathcal{A}$'s view of each execution of Π_{co} by $\mathcal{S}_{\mathsf{co}}$ run on the provider's input and the output proof. A distinguisher between consecutive hybrids breaks CO, so $|\Pr[W_2] - \Pr[W_1]| \le q_s \cdot \mathrm{Adv}^{\mathsf{co}}$. For M1 set Game 2 equal to Game 1.

In Game 2, $\mathcal{A}$'s view consists of pp; values independent of s (uniform shares, the OPRF key, τ, and unlock transcripts, which are functions of π and fresh coins only); $\mathsf{vk}_{\mathsf{in}}$ under V1; the envelopes produced by $\mathcal{O}_U$; and, in M3, simulated transcripts that depend only on the provider's input and the published proof.

Reduction. $\mathcal{B}$ plays $\mathrm{Exp}^{\mathsf{bal}+}$ of [1] with honest-party oracle $\mathcal{O}_s$. It receives pp and $\mathsf{vk}_{\mathsf{in}}$, samples π, an OPRF key, and uniform provider shares itself, and computes τ honestly. It runs $\mathcal{A}$ on this state. On a query to $\mathcal{O}_U$, $\mathcal{B}$ executes the unlock with $\mathcal{A}$ by playing the device side honestly with π (no seed is needed for this), asks $\mathcal{O}_s$ to authorize and publish the requested spend, hands the resulting envelope to $\mathcal{A}$ for relaying, and in M3 runs $\mathcal{S}_{\mathsf{co}}$ on the published proof. $\mathcal{B}$ outputs $\mathcal{A}$'s prefix C. This simulation is exactly Game 2. If W_2 occurs, C contains an accepted spend of an s-owned note not authorized by $\mathcal{O}_s$, which is event (2) of $\mathrm{Exp}^{\mathsf{bal}+}$. By [1, Cor. 2] with the $\mathsf{vk}_{\mathsf{in}}$-augmented theft term, $\Pr[W_2] \le q \cdot \mathrm{Adv}^{\mathsf{ks}}_{R_{\mathsf{tr}}} + \mathrm{Adv}^{\mathsf{auth}+}_{\mathcal{B}}$. Collecting the game hops gives the claim. □

Remark 1 (M2)*.* In M2 the enclave holds the witness. If isolation holds, the enclave is functionally part of the device and Theorem 1 applies with the enclave on the user's side. If it fails, $\mathcal{A}$ learns $\mathsf{sk}_{\mathsf{spend}}$ and no bound is possible; M2 is non-custodial only under the hardware assumption.

6.3 PIN leakage to a co-resident app (E2)

Definition 2 (Co-resident experiment)**.** $\mathrm{Exp}^{\mathsf{app}}_{\mathcal{A}}$ *is* $\mathrm{Exp}^{\mathsf{nc}}_{\mathcal{A},\mathrm{M1}}$ *with V0, except that $\mathcal{A}$ is not the provider: it receives the PIN π in the clear, $\sigma_{\mathcal{P}}$ (as if it had unlocked successfully), the wrapped blob $c_{\mathcal{D}} = \mathsf{AEAD}_{K_{\mathcal{D}}}(\sigma_{\mathcal{D}})$, and arbitrary side-channel observations, but no access to $\mathcal{O}_{\mathsf{KS}}$.*

Theorem 2 (PIN-leak tolerance)**.** *Under KS and SC, for every PPT $\mathcal{A}$ there are PPT $\mathcal{B}_1, \mathcal{B}_2$ with* $\Pr[\mathrm{Exp}^{\mathsf{app}}_{\mathcal{A}} = 1] \le \mathrm{Adv}^{\mathsf{ind}}_{\mathsf{AEAD},\mathcal{B}_1} + q \cdot \mathrm{Adv}^{\mathsf{ks}}_{R_{\mathsf{tr}}} + \mathrm{Adv}^{\mathsf{auth}}_{\mathcal{B}_2}$.

Proof. Game 1 replaces $c_{\mathcal{D}}$ by $\mathsf{AEAD}_{K_{\mathcal{D}}}(0^{256})$. By KS, $\mathcal{A}$ has neither $K_{\mathcal{D}}$ nor oracle access to it, and the honest wallet's own decryptions of $c_{\mathcal{D}}$ during spends are internal and not part of $\mathcal{A}$'s view (oracle spends are simulated through $\mathcal{O}_s$ below). $\mathcal{B}_1$ embeds the IND-CPA challenge ciphertext for messages $(\sigma_{\mathcal{D}}, 0^{256})$ as $c_{\mathcal{D}}$ and simulates the rest of the experiment as in the reduction below. It outputs $\mathcal{A}$'s success bit, so the change in success probability is at most $\mathrm{Adv}^{\mathsf{ind}}_{\mathsf{AEAD}}$.

In Game 1, the only share-dependent value in $\mathcal{A}$'s view is $\sigma_{\mathcal{P}}$, which by Lemma 1 is independent of s. By SC, the side-channel observations depend on π, the layout, and time, but not on s. $\mathcal{B}_2$ then plays $\mathrm{Exp}^{\mathsf{bal}}$ exactly as $\mathcal{B}$ does in Theorem 1, without $\mathsf{vk}_{\mathsf{in}}$ because V0 is used. It samples π, $\sigma_{\mathcal{P}}$, and $c_{\mathcal{D}} \leftarrow \mathsf{AEAD}_K(0^{256})$ under its own key, and produces side-channel observations from π with the same sampler. A win yields an unauthorized accepted spend, bounded by [1, Cor. 2]. □

Theorem 2 formalizes the point of rule R4: a perfect side channel on the PIN does not by itself help the co-resident attacker spend.

6.4 Device theft and the guessing bound

Lemma 2 (Adaptive equality testing)**.** *Let π be distributed on a finite set with posterior $p(\cdot \mid z)$ given side information $Z = z$. Consider any randomized adaptive strategy that makes at most L queries to an oracle $\mathsf{Eq}_\pi(x) = [x = \pi]$ and wins if some answer is* 1*. Then*

$$\Pr[\mathsf{win} \mid Z = z] \le \sum_{i=1}^{L} p_{(i)}(z),$$

where $p_{(1)}(z) \ge p_{(2)}(z) \ge \cdots$ are the posterior probabilities sorted in decreasing order. Querying the L most probable values attains the bound.

Proof. Fix z and the strategy's coins ω, which are independent of π given z. Let $x_1, \ldots, x_L$, each a function of (z, ω), be the queries the strategy makes when every answer is 0; this all-zero path is determined by (z, ω). Let $S(z, \omega) = \{x_1, \ldots, x_L\}$, so $|S| \le L$.

First suppose the strategy wins at query j. All previous answers were 0, so the run up to query j coincides with the all-zero path, hence the winning query is x_j and $\pi = x_j \in S$. Conversely, if $\pi \in S$, let j be the least index with $x_j = \pi$. Answers to $x_1, \ldots, x_{j-1}$ are 0, so the strategy follows the all-zero path to x_j and wins.

Therefore $\Pr[\mathsf{win} \mid z, \omega] = \Pr[\pi \in S(z, \omega) \mid z] = \sum_{x \in S(z,\omega)} p(x \mid z) \le \max_{|S'| \le L} \sum_{x \in S'} p(x \mid z) = \sum_{i \le L} p_{(i)}(z)$. Averaging over ω preserves the bound. Choosing S as the L most probable values gives equality. □

Theorem 3 (Device-theft bound)**.** *Let $\mathcal{A}$ hold the device and pass user presence (the worst case), so that it obtains $\sigma_{\mathcal{D}}$. Let it also hold side information Z jointly distributed with π, such as a fuel-gauge trace. Under GATE, the probability that $\mathcal{A}$ obtains s is at most*

$$G_L := \mathbb{E}_Z\Big[\sum_{i=1}^{L} p_{(i)}(Z)\Big] + \mathrm{Adv}^{\mathsf{oprf}} + 2^{-256}.$$

If user presence is enforced and $\mathcal{A}$ cannot pass it, its success probability is bounded as in Theorem 2, even if π is known exactly.

Proof. Replace the real unlock protocol by $\mathcal{F}_{\mathsf{gate}}$, at cost $\mathrm{Adv}^{\mathsf{oprf}}$. In the ideal world, $\mathcal{A}$ obtains $\sigma_{\mathcal{P}}$ only through a successful Test. Otherwise it holds $\sigma_{\mathcal{D}}$ only, which by the 2-of-3 analogue of Lemma 1 applied to $x_{\mathcal{D}}$ is independent of s. It can then guess s with probability at most 2^{-256}, or $\sigma_{\mathcal{P}}$ given $\sigma_{\mathcal{D}}$ with the same probability, because $\sigma_{\mathcal{P}}$ is uniform given $\sigma_{\mathcal{D}}$ when s is uniform. $\mathcal{F}_{\mathsf{gate}}$ locks after L consecutive failures, and a success already ends the game, so $\mathcal{A}$ makes at most L effective Test queries. Their answers are exactly Eq_π. Lemma 2 bounds the conditional success by $\sum_{i \le L} p_{(i)}(z)$, and taking the expectation over Z gives G_L. The second statement follows because, without user presence, $\mathcal{A}$ is in the position of Theorem 2. □

The simulation in §7 estimates exactly the first term of G_L for a fixed keypad: its attacker ranks candidates by posterior, which Lemma 2 shows is optimal. For randomized keypads the term can be bounded analytically.

6.5 Randomized keypads: an exact bound

Let PINs be strings $\pi \in \Sigma^n$ over $\Sigma = \{0, \ldots, 9\}$, drawn from a prior p supported on a set $\mathcal{P} \subseteq \Sigma^n$. A layout is a bijection $\Lambda : \Sigma \to \mathcal{K}$ onto the ten digit slots $\mathcal{K}$. Rule R1 draws Λ uniformly and independently for each entry, independently of π, of the confirm-key position κ, and of the user's timing noise N. An observation is any randomized function

$$Z = \Phi\big(\Lambda(\pi_1), \ldots, \Lambda(\pi_n), \kappa, N\big),$$

so it may combine timing, current, and motion data, provided they depend on the PIN only through the pressed slots. The *equality pattern* $\mathsf{pat}(\pi)$ is the string obtained by relabelling symbols in order of first occurrence; for example $\mathsf{pat}(7337) = 0110$.

Lemma 3 (Pattern invariance)**.** *If $\mathsf{pat}(\pi) = \mathsf{pat}(\pi')$, then $Z \mid \pi$ and $Z \mid \pi'$ have the same distribution.*

Proof. Because the patterns are equal, the rule $\psi(\pi_i) := \pi'_i$ is well defined: $\pi_i = \pi_j \Leftrightarrow \pi'_i = \pi'_j$. It is injective on the symbols of π, and we extend it to a bijection $\psi : \Sigma \to \Sigma$. If Λ is a uniform bijection $\Sigma \to \mathcal{K}$, so is $\Lambda' := \Lambda \circ \psi$, since composing with a fixed bijection permutes the set of bijections. Hence

$$(\Lambda(\pi'_i))_{i=1}^{n} = (\Lambda(\psi(\pi_i)))_{i=1}^{n} = (\Lambda'(\pi_i))_{i=1}^{n} \stackrel{d}{=} (\Lambda(\pi_i))_{i=1}^{n}.$$

Since (κ, N) is independent of Λ and of the PIN, the joint distribution of the slot sequence together with (κ, N) is the same for π and π', and so is that of Z. □

Theorem 4 (Randomized-keypad guessing bound)**.** *Let $\mathcal{P}$ be partitioned into pattern classes $\mathcal{C}$. Under rule R1, for every observation function Φ and every $L \geq 1$,*

$$\mathbb{E}_Z\Big[\sum_{i=1}^{L} p_{(i)}(Z)\Big] \leq \sum_{C \in \mathcal{C}} \max_{S \subseteq C,\ |S| \leq L} p(S).$$

For the uniform prior on $\mathcal{P}$ the right-hand side equals $\frac{1}{|\mathcal{P}|}\sum_C \min(L, |C|)$. The bound is tight: it is attained by an observer that learns $\mathsf{pat}(\pi)$ exactly.

Proof. By Lemma 3, the likelihood $f(z \mid \pi)$ depends on π only through its class; write $f_C(z)$. With $m(z) = \sum_\pi p(\pi) f(z \mid \pi)$ the marginal density, the posterior is $p(\pi \mid z) = p(\pi) f_C(z)/m(z)$ for $\pi \in C$. Any set T of at most L PINs meets each class C in a set $T \cap C$ of size at most L. Hence

$$\sum_{\pi \in T} p(\pi \mid z) = \sum_C \frac{f_C(z)}{m(z)}\, p(T \cap C) \leq \sum_C \frac{f_C(z)}{m(z)} \max_{S \subseteq C, |S| \leq L} p(S).$$

Taking T to be the L most probable PINs bounds $\sum_{i \leq L} p_{(i)}(z)$. Since

$$\mathbb{E}_Z[f_C(Z)/m(Z)] = \int m(z)\, \frac{f_C(z)}{m(z)}\, dz = \int f_C(z)\, dz = 1,$$

taking expectations gives the stated bound. Under the uniform prior, $\max_{S \subseteq C, |S| \leq L} p(S) = \min(L, |C|)/|\mathcal{P}|$. For tightness, an observer that learns the class C can guess $\min(L, |C|)$ members of it, so it succeeds with probability $\sum_C \Pr[C] \min(L, |C|)/|C| = \sum_C \min(L, |C|)/|\mathcal{P}|$. □

Corollary 1 (Concrete bounds, $L = 10$, uniform prior)**.** *A pattern class with j distinct symbols has $10 \cdot 9 \cdots (10 - j + 1) \geq 10$ members, so each class contributes exactly $L = 10$, and the bound is $10 \cdot B_n/|\mathcal{P}|$ with B_n the number of patterns.*

- *4 digits, all PINs: $B_4 = 15$ (the Bell number), giving $150/10^4 = 1.5\%$, against 0.1% for blind guessing.*
- *4 digits, distinct digits only (R3): one class of size 5040, giving $10/5040 \approx 0.198\%$, which equals blind guessing over that set.*
- *6 digits, all PINs: $B_6 = 203$, giving $2030/10^6 \approx 0.203\%$.*
- *6 digits, distinct digits only: $10/151200 \approx 6.6 \times 10^{-5}$.*

Corollary 1 explains the empirical observation of §7. Randomization caps the side-channel gain at learning the repetition pattern, and the simulated randomized-keypad success rates (0.67–1.0%) lie below the 1.5% cap. With R1 and R3 together, *no* observation of pressed slots—timing, current, or motion—gives any advantage over blind guessing. The guarantee requires a fresh layout per entry: if a layout is reused, observations from several entries are not independent of π given the pattern, and Lemma 3 no longer applies.

6.6 Synchronization integrity (E5)

Let C be the wallet's header chain, obtained by SPV, and h the chosen anchor height. Each indexer $\mathcal{I}_j$ returns a pair (r_j, b_j). The wallet accepts r if at least k indices j satisfy $(r_j, b_j) = (r, \mathsf{hash}_C(h))$. Let B be the set of corrupted indexers. Honest indexers are correct implementations that have replayed a chain containing the block $\mathsf{hash}_C(h)$.

Theorem 5 (Sync integrity and liveness)**.** *Let* $R^* = \mathrm{Replay}(\mathrm{Env}(C))[h]$.

(i) If $|\mathsf{B}| < k$*, any accepted root equals* R^**, except with probability* $\mathrm{Adv}^{\mathsf{cr}}_{\mathsf{hdr}}$ *of a block-header hash collision.*

(ii) If each indexer is independently corrupted with probability p*, then* $\Pr[\text{accept } r \neq R^*] \leq \sum_{i=k}^{n} \binom{n}{i} p^i (1-p)^{n-i} + \mathrm{Adv}^{\mathsf{cr}}_{\mathsf{hdr}}$.

(iii) If at least k *honest indexers respond, the wallet accepts* R^*.

Proof. An honest indexer reporting block hash $\mathsf{hash}_C(h)$ has replayed a prefix whose block at height h has that hash. Barring a header collision, this prefix agrees with C up to h, since each header commits to its predecessor. $R[h]$ depends only on the prefix up to h, because replay at height h reads only envelopes in blocks $\leq h$ [1, §8]. By Shielded Bitcoin Proposition 1 (G1, under A8), every honest indexer therefore returns $r_j = R^*$.

(i) Suppose the wallet accepts $r \neq R^*$. Then k indexers returned r, and none of them is honest by the previous paragraph, so $|\mathsf{B}| \geq k$, a contradiction.

(ii) By (i), a false acceptance requires $|\mathsf{B}| \geq k$. Under independent corruption $|\mathsf{B}| \sim \mathrm{Bin}(n, p)$, which gives the stated tail.

(iii) All honest responses equal $(R^*, \mathsf{hash}_C(h))$, so k of them meet the acceptance threshold. □

For $n = 5$, $k = 3$, $p = 0.1$, the bound in (ii) is 0.00856, against 0.1 for a single trusted indexer. Taking $k > n/2$ also prevents two different roots from both being accepted.

Proposition 1 (Delta download hides note interest)**.** *Under delta download, the sequence of requests a wallet sends to an indexer is a deterministic function of the set of block heights it fetches and of the fetch times. Hence, for any two wallets with the same fetch schedule, the indexer's views are identically distributed, whatever notes the wallets own.*

Proof. Each request is "all appended leaves, all accepted nullifiers, and all output fields of block h". Its content depends only on h. Merkle paths, spent status, and trial decryption are computed locally from the response, so no request depends on wallet secrets or owned positions. The views are therefore equal as functions of the schedule. □

6.7 Recovery (E3)

Theorem 6 (Recovery with share integrity)**.** *(i) If two honest shares are available, reconstruction returns* s*. By [1, Lemma 3], the recovered wallet view differs from* $\mathrm{WalletView}_s(H)$ *with probability at most* $\mathrm{Adv}^{\mathsf{aead}} + \mathrm{Adv}^{\mathsf{cr}}_{H}$*. (ii) If one share holder returns* $\tilde{\sigma} \neq \sigma$*, the fingerprint check fails, except with probability* $\mathrm{Adv}^{\mathsf{cr}}_{H_{\mathsf{fp}}}$ *plus the probability that two seeds derive the same* addr_0*. The wallet then retries with the remaining pair and succeeds by (i).*

Proof. (i) Lagrange interpolation at 0 is exact for polynomials of degree 1: $\lambda_j \sigma_{j,i} + \lambda_{j'} \sigma_{j',i} = s_i(\lambda_j + \lambda_{j'}) + a_i(\lambda_j x_j + \lambda_{j'} x_{j'}) = s_i$. This holds because $\lambda_j + \lambda_{j'} = 1$ and $\lambda_j x_j + \lambda_{j'} x_{j'} = (x_{j'} x_j + x_j x_{j'})/(x_j + x_{j'}) = 0$ in characteristic 2. Shielded Bitcoin recovery from s and replay then applies.

(ii) With $\sigma_{j'}$ honest, the reconstructed byte is $\tilde{s}_i = \lambda_j \tilde{\sigma}_{j,i} + \lambda_{j'} \sigma_{j',i}$. Since $\lambda_j \neq 0$, the map $\tilde{\sigma}_{j,i} \mapsto \tilde{s}_i$ is a bijection, so $\tilde{\sigma} \neq \sigma$ implies $\tilde{s} \neq s$. The check accepts $\tilde{s}$ only if $H_{\mathsf{fp}}(\mathsf{addr}_0(\tilde{s})) = H_{\mathsf{fp}}(\mathsf{addr}_0(s))$. This requires either a collision in H_{fp} or $\mathsf{addr}_0(\tilde{s}) = \mathsf{addr}_0(s)$ for $\tilde{s} \neq s$. The latter is a collision in the HKDF-based derivation chain of Table 1, which is bounded by $\mathrm{Adv}^{\mathsf{cr}}_{H}$ under A4/A5. The fingerprint is not stored at $\mathcal{P}$, so Lemma 1 and Theorem 1 are unaffected. □

6.8 Security of the Rotom share wrap

TBC assumption (TPRP). Let $\mathrm{Perm}(\mathcal{T}, n)$ be the set of families $(P_T)_{T\in\mathcal{T}}$ of permutations of $\{0,1\}^n$. For an adversary $\mathcal{B}$ with forward and inverse oracle access,

$$\mathrm{Adv}^{\pm\mathsf{tprp}}_{\widetilde{E}}(\mathcal{B}) = \Big|\Pr\left[\mathcal{B}^{\widetilde{E}_K,\widetilde{E}_K^{-1}} = 1\right] - \Pr\left[\mathcal{B}^{P,P^{-1}} = 1\right]\Big|,$$

with K uniform and P uniform in $\mathrm{Perm}(\mathcal{T}, n)$. For Rotom-256, $n = 128$ and $|\mathcal{T}| = 2^{128}$. We assume $\mathrm{Adv}^{\pm\mathsf{tprp}}_{\text{Rotom-256}}(\mathcal{B})$ is negligible for every $\mathcal{B}$ running in time well below 2^{240} and making at most 2^{128} queries. This is the provable-security reading of the designers' cryptanalytic claim for the single- and related-tweak settings [10, §2.4]. It is an assumption, as for any concrete block cipher. RW uses one key, so it needs no related-key security, which Rotom does not claim.

Experiments. An adversary $\mathcal{A}$ against RW has two oracles. $\mathsf{Wrap}(\mathsf{ctx}, \sigma)$ returns $\mathsf{RW.Wrap}_K(\mathsf{ctx}, \sigma)$ and may be called at most once per ctx (rule W1). $\mathsf{Ver}(\mathsf{ctx}, \beta)$ returns 1 if $\mathsf{RW.Unwrap}_K(\mathsf{ctx}, \beta) \neq \bot$, else 0; it never returns the plaintext. In the *privacy* game, Wrap returns either real wraps or three independent uniform blocks, and $\mathcal{A}$ must tell which. In the *integrity* game, $\mathcal{A}$ wins if some $\mathsf{Ver}(\mathsf{ctx}, \beta)$ returns 1 for a pair (ctx, β) that Wrap did not output. Plaintexts σ are adversarially chosen in both games, which is stronger than our use, where $\sigma_{\mathcal{D}}$ is a uniform share.

Theorem 7 (Security of RW)**.** *For every $\mathcal{A}$ making q_w wrap and q_v verification queries, there are adversaries $\mathcal{B}_1, \mathcal{B}_2$ making at most $3(q_w + q_v)$ oracle queries, with about the same running time, such that*

$$\mathrm{Adv}^{\mathsf{priv}}_{\mathsf{RW}}(\mathcal{A}) \le \mathrm{Adv}^{\pm\mathsf{tprp}}_{\widetilde{E}}(\mathcal{B}_1), \qquad \mathrm{Adv}^{\mathsf{int}}_{\mathsf{RW}}(\mathcal{A}) \le \mathrm{Adv}^{\pm\mathsf{tprp}}_{\widetilde{E}}(\mathcal{B}_2) + \frac{q_v}{2^n - 1}.$$

For $n = 128$ and $q_v \le F = 10$, the information-theoretic term is below 2^{-124}.

Proof. Step 1: idealize the cipher. $\mathcal{B}_1$ and $\mathcal{B}_2$ run $\mathcal{A}$ and answer each Wrap and Ver query with at most three forward or inverse queries to their own oracle, at the tweaks prescribed by the tweak encoding. With the real cipher they simulate the real game; with a random family P they simulate the game in which $\widetilde{E}_K$ is replaced by P. Each game's success probability therefore changes by at most the respective $\mathrm{Adv}^{\pm\mathsf{tprp}}$. From now on, $\widetilde{E}_K = P$. Because the tweak encoding is injective, the three tweaks $T(\mathtt{enc}, 1)$, $T(\mathtt{enc}, 2)$, and $T(\mathtt{tag}, 1)$ of one context are distinct from each other and from those of every other context. The permutations P_T used by different (call, context) pairs are therefore independent uniform permutations.

Step 2: privacy. By rule W1, each tweak used by Wrap is queried in the forward direction exactly once, at the single point σ_1, σ_2, or $\sigma_1 \oplus \sigma_2$. The image of one point under a uniform permutation is uniform, and the three permutations are independent of each other and of all other wraps. The output (C_1, C_2, Γ) is therefore three independent uniform blocks, whatever σ is. The privacy game has no verification oracle, so the real and random worlds are identical in the ideal-cipher game, and $\mathrm{Adv}^{\mathsf{priv}}_{\mathsf{RW}}(\mathcal{A}) \le \mathrm{Adv}^{\pm\mathsf{tprp}}_{\widetilde{E}}(\mathcal{B}_1)$.

Step 3: integrity. Let G be the ideal game, and G′ the same game except that Ver always answers 0. The two games are identical until the first query that would return 1 in G, so $\Pr[\mathcal{A}\text{ wins in G}] = \Pr[\text{some query of G′ is accepting}]$. In G′, the Ver answers carry no information, so $\mathcal{A}$'s queries depend only on its coins and on the Wrap outputs. We bound the probability that the j-th verification query $(\mathsf{ctx}, C_1', C_2', \Gamma')$ is accepting, conditioned on everything $\mathcal{A}$ has seen. Write $P_1 = P_{T(\mathtt{enc},1)}$, $P_2 = P_{T(\mathtt{enc},2)}$, and $P_\gamma = P_{T(\mathtt{tag},1)}$ for this context; the query is accepting iff

$$P_1^{-1}(C_1') \oplus P_2^{-1}(C_2') = P_\gamma^{-1}(\Gamma'). \tag{1}$$

Case (a): ctx was never wrapped.
Then $\mathcal{A}$'s view in G' is independent of P_1, P_2, P_γ, because wraps of other contexts use other, independent permutations. Any fixed point's preimage under a uniform permutation is uniform, so $P_\gamma^{-1}(\Gamma')$ is uniform and independent of $P_1^{-1}(C_1') \oplus P_2^{-1}(C_2')$. Equation (1) therefore holds with probability exactly 2^{-n}.

Case (b): ctx was wrapped as (C_1, C_2, Γ) for (σ_1, σ_2), and $(C_1', C_2', \Gamma') \neq (C_1, C_2, \Gamma)$.
$\mathcal{A}$'s view constrains each of the three permutations at exactly one point: $P_1(\sigma_1) = C_1$, $P_2(\sigma_2) = C_2$, and $P_\gamma(\sigma_1 \oplus \sigma_2) = \Gamma$.

- If $C_1' \neq C_1$, then given the view, $P_1^{-1}(C_1')$ is uniform on $\{0,1\}^n \setminus \{\sigma_1\}$ and independent of P_2, P_γ. Equation (1) fixes it to one value, so it holds with probability at most $1/(2^n - 1)$.
- If $C_1' = C_1$ and $C_2' \neq C_2$, the same argument applies to P_2.
- If $C_1' = C_1$ and $C_2' = C_2$, then $\Gamma' \neq \Gamma$. The left side of (1) equals $\sigma_1 \oplus \sigma_2$, while $P_\gamma^{-1}(\Gamma') \neq \sigma_1 \oplus \sigma_2$ because P_γ is a bijection and $P_\gamma(\sigma_1 \oplus \sigma_2) = \Gamma \neq \Gamma'$. The probability is 0.

Every verification query is thus accepting with probability at most $1/(2^n - 1)$ given $\mathcal{A}$'s view in G'. A union bound over the q_v queries gives at most $q_v/(2^n - 1)$, and adding Step 1 proves the integrity bound. □

Corollary 2 (Theorem 2 in the hardware variant)**.** *In the hardware variant, replace the keystore assumption KS by the assumption that K_{HW} never leaves the chip. Then Theorem 2 holds with $\mathrm{Adv}^{\mathsf{ind}}_{\mathsf{AEAD}}$ replaced by $\mathrm{Adv}^{\pm\mathsf{tprp}}_{\text{Rotom-256}}$ for $3(q_w + q_v)$ queries. In addition, an attacker who modifies the stored blob, for example to make the chip release a share of its choosing, succeeds with probability at most $\mathrm{Adv}^{\pm\mathsf{tprp}}_{\text{Rotom-256}} + F/(2^{128} - 1)$.*

Proof. In the proof of Theorem 2, Game 1 replaces the wrapped share by an encryption of zeros. By Theorem 7 (privacy), the blob (C_1, C_2, Γ) is indistinguishable from three uniform blocks, which are independent of $\sigma_{\mathcal{D}}$, at cost $\mathrm{Adv}^{\pm\mathsf{tprp}}$. The rest of the proof is unchanged. For the second statement, a modified blob is accepted only if some Ver query in the integrity game is accepting. Rule W3 caps the number of rejected attempts at F before the key is erased, so $q_v \leq F$, and Theorem 7 (integrity) applies. □

Proposition 2 (Bounded chosen-input traces)**.** *Under rules W1–W3, a physical attacker holding the chip obtains power traces of Rotom evaluations on at most F distinct attacker-chosen ciphertext blocks per tweak before K_{HW} is erased. Every other unwrap evaluates Rotom on the single stored blob.*

Proof. The chip evaluates Rotom with K_{HW} only in Wrap, at most once per context by W1 and on plaintext the chip generated itself, and in Unwrap. The tweak is chip-internal, so the attacker controls only the ciphertext input to Unwrap. An input equal to the stored blob is the same value on every call. Any other input is a modified blob, which is rejected except with the probability of Corollary 2, and W3 erases the key after F rejections. The number of distinct chosen inputs is therefore at most F. □

Proposition 2 is a statement about *data diversity*, not a proof of physical security. Correlation power analysis of the kind simulated for Rotom [10, §5.4.2] needs many traces with varying known inputs; in that simulation, unmasked Rotom fell to about 500 traces, while W3 allows $F = 10$. Repeated traces of the stored blob let an attacker average out noise but provide no input variation. Profiled template attacks and fault injection remain possible, so masking and standard fault countermeasures are still required for chips exposed to physical attack.

6.9 What the provider learns

Proposition 3 (Provider view in M1 + V0 + anonymous relay)**.** *Assume envelopes reach the relayer through a channel that is unlinkable to the user's account (ANON). Then the provider's*

view of a user can be simulated from the user's unlock timestamps alone, up to $u \cdot \mathrm{Adv}^{\mathsf{obl}}$ *for* u *unlocks.*

Proof. Per unlock, the provider sees a blinded OPRF element, a tag τ', a fresh device ephemeral key, and a timestamp. By obliviousness, the blinded element is simulatable without π, at cost $\mathrm{Adv}^{\mathsf{obl}}$ per unlock. For an honest user $\tau' = \tau$, which the provider already stored at registration. The ephemeral key is fresh and independent of everything else. Under V0 the provider receives no viewing key. Under ANON it cannot attribute relayed envelopes to the account, and the envelopes themselves are public. The simulator therefore outputs τ, fresh blinded elements, and fresh ephemeral keys at the given times. □

Theorem 8 (Graph reconstruction under V1 + authenticated relaying)**.** *Suppose* $\mathcal{P}$ *serves users* $U_1, \ldots, U_m$*, holds* $\mathsf{vk}_{\mathsf{in}}^{(b)}$ *for each (V1), and relays their envelopes under their authenticated accounts. Consider the algorithm* Graph*: for each relayed envelope* e *from* U_a*, each output* j *of* e*, and each* b*, trial-decrypt* $(\mathsf{pk}_{\mathsf{eph},j}, \mathsf{ct}_j^{\mathsf{note}})$ *with* $\mathsf{vk}_{\mathsf{in}}^{(b)}$*, apply the checks of [1, §16.1], and on success output* (U_a, U_b, v)*. For every honestly generated accepted envelope,* Graph *outputs exactly the true recipient and value of each output whose recipient is served by* $\mathcal{P}$*, except with probability at most* $Mm \cdot \mathrm{Adv}^{\mathsf{aead}}$*, where* M *is the number of outputs.*

Proof. Authenticated relaying identifies the sender U_a. Fix an output j paid to a served user U_b. By correctness of recipient encryption (A6), decryption under $\mathsf{vk}_{\mathsf{in}}^{(b)}$ returns the encrypted $(v, d, r_{\mathsf{seed}})$, and the consistency checks pass because the envelope is honest. For $b' \neq b$, passing the checks would mean that a single $(\mathsf{pk}_{\mathsf{eph}}, \mathsf{ct}^{\mathsf{note}})$ opens validly under two recipient keys, which the key-committing and note-opening-robust encryption of A6 excludes except with $\mathrm{Adv}^{\mathsf{aead}}$. A union bound over the at most Mm (output, user) pairs gives the claim. Outputs to unserved recipients fail all trials, so the set of outputs Graph labels is exactly the set paid to served users. □

Theorem 8 does not contradict the transcript privacy of Shielded Bitcoin [1, Thm. 2]. The attack uses only capabilities that lie outside that theorem's leakage boundary: disclosed viewing keys and an authenticated publication channel [1, §20.5, §20.7]. Its effect also reaches users outside $\mathcal{P}$. Every note known to $\mathcal{P}$ is removed from the plausible spend candidates of other users' transfers, shrinking their effective privacy sets [1, §20.6]. Comparing Proposition 3 with Theorem 8 shows that the scanning mode and the relaying channel, not the cryptography, decide whether the provider learns timestamps or the whole payment graph. SBEW therefore defaults to V0 and requires anonymous relaying whenever V1 is enabled.

6.10 Leakage by mode

Table 3 lists $\mathsf{Leak}_{\mathsf{mode}}$ beyond the Shielded Bitcoin transcript leakage $\mathsf{Leak}_{\mathsf{SB}}$ (arity, anchor age, timing, fees, boundary events [1, §20.3–20.5]).

Even in the best mode, PIN unlocks reveal *when* a user spends. Unlock events correlate with envelope publication times; batching unlocks (e.g. releasing $\sigma_{\mathcal{P}}$ for a short session) or random delays before publication weakens the correlation.

7 Evaluation: Fuel-Gauge Leakage on PIN Entry

Giraud and Naccache demonstrate the channel but do not measure how much it narrows the PIN space [2]. We model it to choose the UI rules R1–R4 and to evaluate the first term of G_L in Theorem 3 for fixed keypads, where no analytic bound exists.

Mode	Additional leakage to $\mathcal{P}$	Custody assumption
M1 + V0 + anon. relay	online times; PIN-unlock times (spend attempts)	threshold only
M1 + V1	all incoming notes (value, timing) of the user	threshold only
M1 + V1 + auth. relay	payment graph among $\mathcal{P}$'s users (Thm. 8)	threshold only
M2	full witness if enclave fails	enclave isolation
M3	spend timing; MPC transcript (simulatable)	threshold + MPC security

Table 3: Leakage and trust of SBEW deployment modes. Indexers learn only online times under delta download.

Keypad	f	trials	hit@1	hit@10 (95% CI)	median rank	2^H
blind guess	—	—	0.01%	0.10%	5000	10000
fixed	4 Hz	4000	0.10%	1.03% (0.76–1.39)	1468	5491
fixed	10 Hz	4000	0.33%	2.97% (2.49–3.55)	696	3307
fixed	50 Hz†	4000	0.33%	3.38% (2.86–3.98)	466	2645
randomized (digits)	10 Hz	300	0.00%	0.67% (0.18–2.40)	2745	6073
randomized (digits+OK)	10 Hz	800	0.13%	1.00% (0.51–1.96)	2336	6108

Table 4: PIN recovery under the fuel-gauge timing model, $\sigma_h = 0.15$. †Faster than the 4–10 Hz refresh reported in [2]; shown as an upper reference. Randomized rows use fewer trials because the Bayes-optimal attacker is costlier; their intervals are correspondingly wider.

Model. The keypad is a 3×4 grid (`1 2 3 / 4 5 6 / 7 8 9 / DEL 0 OK`) with unit key pitch. The time between presses at distance D follows Fitts' law [13], $t = a + b\log_2(1 + D/W)$ with a=0.25 s, b=0.15 s/bit, W=1, multiplied by lognormal noise with parameter σ_h that captures a user's inconsistency. The fuel gauge samples at f Hz with random phase, so each press is observed only as the sample index in which it falls; we assume, as is pessimistic for the defender, that every press is detected. The adversary observes the four intervals (digit$_1$ →digit$_2$, …, digit$_4$→`OK`), knows the timing model, and ranks all 10^4 PINs by likelihood under a uniform prior. For randomized layouts it computes the Bayes-optimal likelihood by averaging over random layouts; this likelihood depends only on the PIN's digit-equality pattern. We report success within L guesses, median rank, and the effective candidate set $2^{H(P)}$ [14]. Appendix A lists every model parameter; the simulation code and raw results are available from the authors on request.

Results. Table 4 and Figure 1 show three effects.

1. **The channel is real but bounded by human noise.** On a fixed keypad at 10 Hz, ten guesses succeed in about 3% of entries for σ_h=0.15, a thirty-fold gain over blind guessing (0.1%), and about 6.7% for a very consistent user (σ_h=0.05, median rank 200). At 4 Hz the gain is roughly ten-fold. This is far from the near-unique recovery suggested by the worked example in [2], which used noiseless delays, but against an L=10 attempt limit it is a material increase in a thief's success probability.
2. **Faster sampling helps little beyond 10 Hz**, because human variability rather than quantization dominates: hit@10 rises only from 2.97% to 3.38% between 10 and 50 Hz.
3. **Randomized layouts remove distance information but not repetition.** With per-entry randomization, the attacker's posterior depends only on which digits repeat, and success returns close to the blind level. In a separate run of 600 randomized entries (σ_h=0.05, 10 Hz), all three ten-guess successes were PINs containing a repeated digit; none of the 288 PINs with four distinct digits was found. A repeated digit produces a zero-distance interval regardless

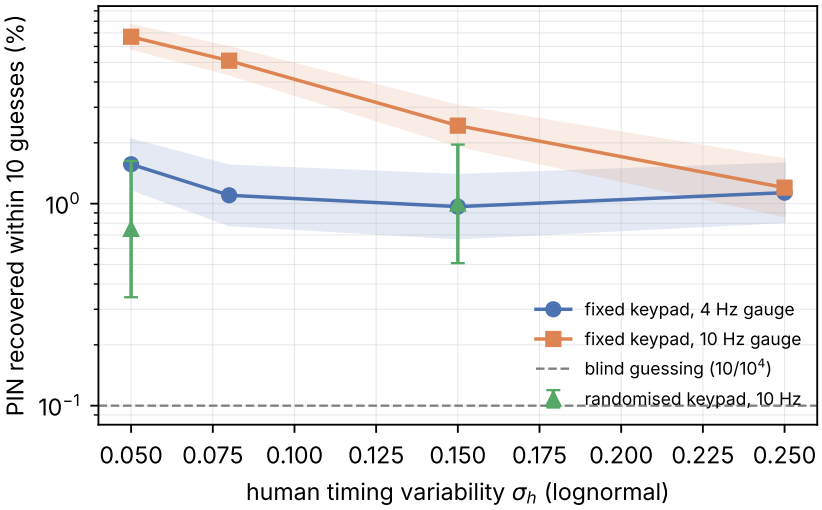


Figure 1: Probability of recovering a 4-digit PIN within ten guesses as a function of user timing variability. Shaded bands and error bars are 95% Wilson intervals.

of layout. Rule R3 addresses this.

Effect on Theorem 3. With L=10, a device thief who also has a fuel-gauge trace succeeds with roughly 3–7% probability on a fixed keypad, versus about 0.1% blind. With R1 the advantage is provably capped at 1.5% for any observation of pressed keys (Corollary 1), consistent with the simulated 0.67–1.0%. With R1 and R3 it is exactly the blind rate. The device share's user-presence binding removes the scenario entirely when it is enforced, which is why R4 and keystore gating are part of the method rather than optional.

Model limitations. The timing parameters are illustrative, not fitted to measured traces; real users have digit-specific habits that a profiled attacker could exploit, which would increase leakage on fixed keypads. We assume perfect press detection; disabling haptics (R2) makes detection harder in practice [2]. The model ignores gyroscope fusion, which [2] uses to sharpen timing and which can leak spatial information directly [15, 16]. On-device measurement across devices is future work.

8 Discussion

Why not threshold signing? The cleanest embedded-wallet custody would let device and provider jointly authorize spends with FROST [4] while a delegated prover sees no spend secret. Shielded Bitcoin binds spending to knowledge of $\mathsf{sk}_{\mathsf{spend}}$ inside the proof, so this requires a split of authority in the style of Zcash Sapling, where a proof-authorizing key enables proving and a re-randomized key signs a spend authorization outside the circuit [9]. Adding this to Shielded Bitcoin would change the transfer statement and therefore require a new, non-interoperable deployment profile and setup [1, App. A]. It would also make M2-style delegated proving safe for custody, though not for privacy. We regard it as the most valuable protocol extension for embedded deployments and leave its specification and analysis to future work.

Viewing keys and compliance. The Shielded Bitcoin disclosure model already gives the user scoped read capabilities [1, §7]. An embedded provider must not make $\mathsf{vk}_{\mathsf{in}}$ or $\mathsf{vk}_{\mathsf{out}}$ escrow a condition of service; when a regulated provider needs evidence, NoteOpening and

DerivedReport disclosures, or the optional certified-recipient lineage of [1, App. D], reveal less than a raw viewing key.

Liveness. Every spend in SBEW requires $\mathcal{P}$'s HSM to release $\sigma_{\mathcal{P}}$, and M3 requires $\mathcal{P}$ online. If $\mathcal{P}$ disappears, the user recovers with $\{\sigma_{\mathcal{D}}, \sigma_{\mathcal{R}}\}$ and continues as a self-custodial Shielded Bitcoin wallet; funds are never stranded by the provider, and anchor-window expiry only forces a rebuild, not a loss [1, §13].

Limitations. The Rotom wrap's security reduces to Rotom-256 being a tweakable PRP. That is a cryptanalytic claim by its designers, not a proven property, and a masked Rotom implementation has not yet been evaluated [10, §5.4.2]; the cycle costs in §4.7 are summed from the designers' per-component figures rather than measured for the wrap itself. We do not measure on-device or MPC proving cost because the Shielded Bitcoin circuit and curve parameters are not yet fixed. Our proofs reduce to the Shielded Bitcoin theorem and to idealized models of the keystore (oracle access only) and of the PIN gate (an ideal functionality realized by an OPRF); attacks on these components, such as keystore extraction, are outside the proofs. The side-channel evaluation is a model, not a device experiment.

9 Conclusion

Shielded Bitcoin specifies a private transfer layer for Bitcoin but leaves the wallet to the deployer. For embedded wallets, the decisive questions are where $\mathsf{sk}_{\mathsf{spend}}$ lives, who sees the witness, who learns which notes are whose, and how much the unlocking PIN leaks. SBEW answers them without touching the transfer statement: a 2-of-3 seed split with an attempt-limited OPRF PIN gate, on-device or collaborative proving, k-of-n root agreement with delta download, local scanning by default, and side-channel-hardened PIN entry. The analysis shows that the largest risk is not cryptographic but organizational: a provider that scans and relays for its users can reconstruct their payment graph. The simulation shows that fuel-gauge leakage is modest but meaningful against attempt-limited PINs, and that keypad randomization neutralizes it except for repeated digits.

A Simulation Details

This appendix gives the complete model behind Table 4 and Figure 1 so that the results can be reproduced.

Keypad. Slots lie on a 4×3 grid with unit pitch. Row $r \in \{0, 1, 2, 3\}$ and column $c \in \{0, 1, 2\}$ give coordinates (r, c). Digits 1–9 occupy rows 0–2 in reading order, digit 0 is at $(3, 1)$, `DEL` is at $(3, 0)$, and `OK` is at $(3, 2)$. In the "randomized (digits)" layout, the ten digits are permuted uniformly over their ten slots on each entry. In "randomized (digits+OK)", `OK` and `DEL` additionally swap positions with probability 1/2.

Timing. The interval between consecutive presses at Euclidean distance D is $t = (a + b \log_2(1 + D/W)) \cdot e^{\eta}$ with $a = 0.25\,\mathrm{s}$, $b = 0.15\,\mathrm{s/bit}$, $W = 1$, and $\eta \sim \mathcal{N}(0, \sigma_h^2)$ drawn independently for each interval. We use $\sigma_h \in \{0.05, 0.08, 0.15, 0.25\}$ and report $\sigma_h = 0.15$ in Table 4.

Observation. Press times are cumulative sums of the four intervals ($\mathrm{digit}_1 \to \mathrm{digit}_2$, $\mathrm{digit}_2 \to \mathrm{digit}_3$, $\mathrm{digit}_3 \to \mathrm{digit}_4$, $\mathrm{digit}_4 \to$ `OK`), starting at 0. Each press is mapped to the index $\lfloor (t + \phi) f \rfloor$ of the fuel-gauge sample in which it falls, where $\phi \sim U[0, 1/f)$ is a random phase per entry and $f \in \{4, 10, 50\}\,\mathrm{Hz}$. The adversary observes the four differences of consecutive indices.

Adversary. For each distinct distance value, the likelihood of an observed index difference is estimated from 2×10^5 Monte Carlo samples with independent phase, with a floor of 10^{-12} on empty bins. For the fixed layout, a PIN's log-likelihood is the sum over the four intervals, and all 10^4 PINs are ranked under the uniform prior, with ties broken uniformly at random. For randomized layouts, the likelihood of each of the 15 equality patterns is the average over 150 sampled layouts. By Lemma 3 this is the exact form of the Bayes-optimal likelihood, up to Monte Carlo error, and all PINs in a pattern share it.

Metrics and statistics. For each entry we record the rank of the true PIN. Hit@L is the fraction of entries with rank at most L. 2^H is the mean over entries of $2^{H(P)}$ for the posterior P. Confidence intervals are 95% Wilson intervals. Trials per configuration are 4000 (fixed layout, Table 4), 3000 (fixed layout, Figure 1), and 300 or 800 (randomized). The pseudorandom generator is NumPy's `default_rng` with seed 20261105.

B Notation

Symbol	Meaning
$s = \mathsf{seed}_{\mathsf{wallet}}$	32-byte wallet seed of Shielded Bitcoin
$\sigma_{\mathcal{D}}, \sigma_{\mathcal{P}}, \sigma_{\mathcal{R}}$	device, provider, and recovery shares of s
π, L	user PIN; HSM attempt limit
$\mathcal{F}_{\mathsf{gate}}$	ideal PIN-gate functionality
$K_{\mathcal{D}}$, K_{HW}	keystore key (phone); chip-unique key (hardware variant)
φ	address fingerprint for share-integrity checks
$\mathsf{sk}_{\mathsf{spend}}, \mathsf{sk}_{\mathsf{nf}}, \mathsf{vk}_{\mathsf{in}}, \mathsf{vk}_{\mathsf{out}}$	Shielded Bitcoin capabilities (Table 1)
M1, M2, M3	on-device, enclave, collaborative proving
V0, V1	local and delegated scanning
R1–R4	PIN-entry rules
W1–W3	hardware-wrap rules
RW, $\widetilde{E}$	Rotom share wrap; Rotom-256
$R[h]$, R^*	replayed root at height h; correct root
n, k	number of indexers queried; agreement threshold
$\mathsf{pat}(\pi)$	equality pattern of a PIN
G_L	device-theft success bound (Theorem 3)

References

[1] C. Shikhelman, M. Komarov, A. Moskvin. *Shielded Bitcoin: Private Transfers on the Bitcoin L1.* Cryptology ePrint Archive, Paper 2026/2749, 2026.

[2] V. Giraud, D. Naccache. *Bypassing Android isolation with fuel gauges: new risks with advanced power ICs.* Cryptology ePrint Archive, Paper 2023/1078, 2023.

[3] R. Gennaro, S. Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. In *ACM CCS*, 2018.

[4] C. Komlo, I. Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *Selected Areas in Cryptography (SAC)*, 2020.

[5] S. Jarecki, H. Krawczyk, J. Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In *EUROCRYPT*, 2018.

[6] A. Shamir. How to share a secret. *Communications of the ACM*, 22(11), 1979.

[7] A. Ozdemir, D. Boneh. Experimenting with collaborative zk-SNARKs: Zero-knowledge proofs for distributed secrets. In *USENIX Security*, 2022.

[8] G. Beck, J. Len, I. Miers, M. Green. Fuzzy message detection. In *ACM CCS*, 2021.

[9] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox. *Zcash Protocol Specification*. Electric Coin Company, 2016–2025.

[10] K. Morita, K. Minematsu, K. Sakamoto, K. Taka, R. Ueno, T. Isobe. *Rotom: A Microcontroller-Oriented Tweakable Block Cipher Using Modular Multiplication* (full version). Cryptology ePrint Archive, Paper 2026/2693, 2026.

[11] M. Liskov, R. L. Rivest, D. Wagner. Tweakable block ciphers. In *CRYPTO*, 2002.

[12] T. Krovetz, P. Rogaway. The software performance of authenticated-encryption modes. In *FSE*, 2011.

[13] P. M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6), 1954.

[14] A. Serjantov, G. Danezis. Towards an information theoretic metric for anonymity. In *Privacy Enhancing Technologies (PET 2002)*, LNCS 2482, Springer, 2003.

[15] Z. Xu, K. Bai, S. Zhu. TapLogger: Inferring user inputs on smartphone touchscreens using on-board motion sensors. In *ACM WiSec*, 2012.

[16] D. Berend, B. Jungk, S. Bhasin. There goes your PIN: Exploiting smartphone sensor fusion under single and cross user setting. Cryptology ePrint Archive, Paper 2017/1169, 2017.